Reproducible Answer Selection over Large Language Model Ensembles, with an Exact Kemeny Reference and a Ground-Truth Audit

Aris Dressino[a,*]

[a] *TUM School of Management, Technical University of Munich, Arcisstraße 21, Munich, 80333, Germany*

Abstract

Sampling several answers from a large language model and choosing one buys accuracy at inference time, but the choosing rule is either majority vote, which fails when answers do not repeat, or a language-model judge whose verdicts are not reproducible. We cast selection as an integer program over a judge-derived pairwise matrix and a deterministic self-consistency term, so that the winner is exactly reproducible given the judge's outputs. The Copeland-style aggregate is a 5-approximation to the NP-hard Kemeny-optimal ranking, and we report the exact Kemeny–Young winner as a reference. We audit the method against self-consistency and greedy sampling on six benchmarks (532 mathematics, science and knowledge problems), with paired exact tests and a headroom-and-recovery metric. The audit is negative. The judge scores 60.5% on RewardBench. With homogeneous generation the method is indistinguishable from self-consistency (10 versus 13 discordant problems of 532); with heterogeneous generation it is significantly worse (10 versus 32 of 402, $p < 0.001$). The default cost term dominates the quality term in 653 of 729 replayed problems, and the judge concentrates its picks on one generator (75.6%) that is no more accurate. No other judge-based selector we tried beats plurality voting. A closed-form threshold shows why: with three candidates of 80% accuracy, a judge must prefer the correct

*Corresponding author. ORCID: 0009-0003-0693-9051.
Email address: aris.dressino@tum.de (Aris Dressino)

answer in more than 80% of mixed pairs, and we measure 49–56% on every track with at least 70 mixed pairs. Reproducible selection is not accurate selection; we release the protocol and replayable data.

Keywords: Artificial intelligence in OR, Large language models in OR, Integer programming, Group decisions and negotiations, Decision support systems

---

1. Introduction

Large language models are sampled, not computed: even greedy decoding is not reproducible across runs or hardware because of GPU batch-invariance and floating-point non-associativity (Atil et al., 2024). The usual response is to draw N candidate answers and pick one. Self-consistency (Wang et al., 2023b) picks the majority final answer; it is well-defined only when candidates cluster, and on open-ended queries it degenerates to an arbitrary tie-break (in our open-ended pilot, zero of $N = 5$ candidates matched exactly on every query). Universal self-consistency (Chen et al., 2023) and judge-based selection replace the vote by the choice of a large language model (LLM), at the price of a selection step that is itself a stochastic LLM call.

We do not try to make generation deterministic. We treat *selection* over an already-sampled set as an optimization problem: an LLM judge produces a pairwise preference matrix, and an integer program combines it with a deterministic self-consistency signal, a confidence estimate and a cost penalty to pick exactly one winner. We call the resulting selector OR-Select. Given fixed candidates and fixed judge outputs, the winner is exactly reproducible. What is not automatically reproducible, or even accurate, is the judge; we therefore evaluate the selector against ground truth rather than assume that a principled aggregation rule translates into better answers (Landesberg, 2026).

Contributions.

1. **Formulation and replayability.** An integer-programming formulation of best-of-N selection that treats self-consistency as one deter-

ministic term next to an LLM-judged pairwise term, together with a storage format that makes every selection exactly replayable offline (Section 3.2).

2. **Reference solutions.** The tractable Copeland-style aggregate inherits the 5-approximation guarantee of Coppersmith et al. (2010) against the NP-hard Kemeny-optimal ranking; we additionally compute the exact Kemeny–Young winner on every problem and report how often, and in which direction, the two differ (Sections 3.4 and 6.8). The approximation result is not ours; its empirical consequence for LLM judging is.
3. **An audit protocol and open data.** Paired exact tests against self-consistency, greedy and exact Kemeny–Young; a floor/oracle *recovery* metric that separates selector quality from how much there was to select; valid/total coverage reporting with a flagged, higher-cap second pass for truncated generations; a judge-only validation on RewardBench; and offline weight ablations that cost no API calls because the judge outputs are stored (Sections 5 and 6).
4. **Pairwise-accuracy thresholds.** A closed-form condition for $N = 3$ (Theorem 4.1) for when a pairwise judge with Copeland aggregation beats plurality voting; exact thresholds for $N \leq 7$, pair-dependent judges and correlated wrong answers; a large-N lower bound (Proposition S1); all confronted with the measured pairwise accuracy of the judge (Section 6.7).
5. **Findings.** (a) The judge is only moderately reliable (60.5% on RewardBench). (b) With homogeneous generation, judge-based selection does not beat majority vote. (c) With heterogeneous generation it is significantly worse when pooled (not on GPQA Diamond alone), and the mechanism is identifiable: a length-scale mismatch in the default objective plus a generator-specific preference of the judge. (d) The exact Kemeny–Young winner differs from the shipped selector's winner on 139 of 854 problems but is not more accurate.

The negative result is the point of contribution 5: judge-based aggre-

gation is attractive in theory, and the literature on correlated judge errors (Kohli, 2026; Kim et al., 2025) and on the weak link between judge metrics and selection accuracy (Landesberg, 2026) predicts trouble; this paper measures it on six benchmarks with an exact reference and explains where it comes from.

2. Related Work

Selection among sampled answers. Self-consistency (Wang et al., 2023b) votes over extracted answers; universal self-consistency (Chen et al., 2023) lets the model pick the most consistent free-form response. Pairwise approaches are closest to ours: PAIRS (Liu et al., 2024) finds that pairwise comparison plus rank aggregation beats asking an evaluator to score or select directly, LLM-Blender (Jiang et al., 2023) ranks several models' outputs by pairwise comparison, and LLM-PeerReview (Chen et al., 2025) aggregates several judges by averaging or truth inference. Repeated sampling raises coverage far faster than selectors can exploit it, so the bottleneck is choosing the sample, not producing it (Brown et al., 2024; Snell et al., 2024); verifier-based selection (Cobbe et al., 2021; Lightman et al., 2023) needs a trained model, whereas our setting is training-free. Our floor/oracle recovery metric (Section 5.5) makes the coverage-versus-selection gap explicit per problem.

Judge reliability and correlated errors. LLM judges show positional bias (Zheng et al., 2023; Wang et al., 2023a), self-preference (Panickssery et al., 2024; Wataoka et al., 2024) and other inconsistencies (Gu et al., 2024); permutation self-consistency (Tang et al., 2024) reduces the first by averaging over orderings, and we adopt its pairwise form. Adding judges or models does not remove the problem when errors are shared: nine judges carry about two effective votes, and established aggregators close at most 11% of the gap (Kohli, 2026); larger models make more correlated errors (Kim et al., 2025); agreement is a weak correctness signal on GPQA Diamond and AIME (Ding, 2026); and a judge with moderate global correlation recovers only about a fifth of the gain of perfect selection (Landesberg, 2026),

whose recommendation to report within-prompt selection quality our recovery metric follows. Yang et al. (2025) find that Kemeny–Young aligns best with human rankings when aggregating peer evaluations into *model* rankings; we use this as motivation for the Kemeny reference, not as evidence about answer selection.

Rank aggregation and voting theory. The Kemeny–Young rule (Kemeny, 1959; Young & Levenglick, 1978) is NP-hard to compute exactly (Bartholdi et al., 1989), which motivates tractable surrogates (Dwork et al., 2001); ordering by weighted wins is a 5-approximation (Coppersmith et al., 2010), the result Proposition 3.1 uses. It is the maximum-likelihood ranking under a Condorcet noise model in which every pairwise comparison is independently correct with the same probability above one half (Young, 1988), which is exactly the judge model of Theorem 4.1; other rules are maximum-likelihood under other noise models (Conitzer & Sandholm, 2005), and sample complexity depends on rule and noise (Caragiannis et al., 2016). Condorcet's jury theorem weakens when votes are correlated (Ladha, 1992; Dietrich & List, 2004), a point applied to LLM ensembles (Lefort et al., 2024; Kohli, 2026). For reward-model best-of-N against majority voting, asymptotic and minimax analyses exist (Di et al., 2025; Komiyama et al., 2025), a preference-graph method proves recovery when evaluators are accurate enough (Hu et al., 2024), and second-order information improves on plain voting (Ai et al., 2025). Our contribution is not the noise model but a finite-N, checkable condition on the measurable pairwise accuracy q under which the pairwise route beats plurality of answers, together with the measured values.

Operational-research precedents. Three streams of this journal's literature meet in our problem. *Exact and heuristic Kemeny aggregation*: exact and metaheuristic methods for the median ranking, including weak and partial rankings (Amodio et al., 2016; Aledo et al., 2018; Azzini & Munda, 2020), and faster computation by a necessary condition for being a winner (Rico et al., 2023). Our exact reference is a small integer program solved with the constraint-programming solver CP-SAT at $N \leq 6$ and we claim no faster

solver; what we take from this literature is the separation of the *winner* of a Kemeny aggregation from the full ranking, which Theorem 4.1 and Section 6.8 make operational for LLM judges. *Ensemble selection as optimization*: De Bock et al. (2020) and Aktaş et al. (2024) select which models to include in an ensemble; we optimize which answer to return for each problem, with a cost term that proves decisive (and harmful). *Judgment and voting*: crowd and judgment aggregation (Brown & Reade, 2019; Jaspersen, 2022), a machine-learning approach to voting rules (Burka et al., 2022) and human judgment in selecting among forecasts (De Baets & Harvey, 2020) treat "who or what picks among alternatives" as an object of study, and fused LLMs have recently been applied to prediction tasks in this journal (Maarouf et al., 2025). We found no paper in this journal that audits LLM answer selection against ground truth, and we make no claim that the ingredients (pairwise judging, Copeland aggregation, Kemeny–Young) are new. An audit on 31,900 ensemble subsets finds that simple voting beats the strongest member in about 10% of size-3 subsets (Kim, 2026); ours holds the *selector* to ground truth, with an exact Kemeny reference, replayable judge outputs and a closed-form accuracy threshold.

3. Method

3.1. Pipeline

Given a query, the pipeline runs five stages (Figure 1).

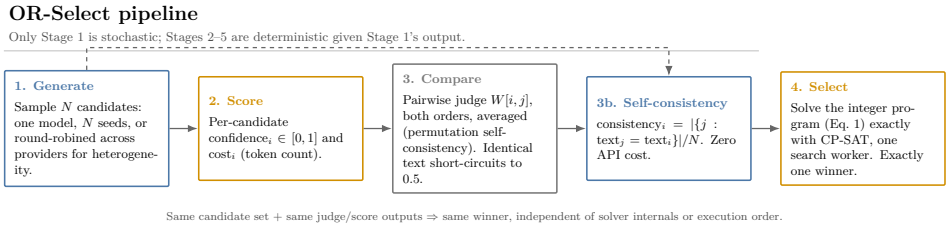


Figure 1: The OR-Select pipeline. Generation (Stage 1) is the only stochastic stage; every later stage is deterministic given Stage 1's output and the judge's outputs. Stage 3b runs in parallel with Stage 3, from the same candidate texts, at zero additional API cost.

Generate. Sample N candidate answers, either from one model with N distinct seeds or round-robined across several models for heterogeneity. Each candidate must satisfy a domain-specific validity predicate (for example, an extractable answer letter on GPQA): a non-empty completion can still be unusable. On failure the slot is resampled with a freshly varied seed, up to three times; a problem whose slot never yields a valid answer is reported as ungraded rather than scored on invalid text.

Score. Obtain a per-candidate confidence $\text{confidence}_i \in [0, 1]$ and a cost proxy cost_i, the candidate's length in whitespace-separated words.

Compare. Build a pairwise preference matrix $W[i, j]$: for each unordered pair an LLM judge is asked twice, once per text order, and the two votes averaged, following permutation self-consistency (Tang et al., 2024) to cancel positional bias (Zheng et al., 2023; Wang et al., 2023a) (Figure S2). The reverse entry is $W[j, i] = 1 - W[i, j]$, never re-judged. Pairs with byte-identical candidate text skip the judge and receive $W[i, j] = 0.5$ (Section S6 explains why this matters beyond efficiency). Judge calls scale as $N(N-1)$ in the worst case, the dominant cost of the pipeline.

Self-consistency. Compute $\text{consistency}_i = |\{j : \text{text}_j = \text{text}_i\}|/N$ by string comparison: zero API calls, and self-consistency voting (Wang et al., 2023b) as one signal rather than the whole decision rule.

Select. Solve the integer program below exactly with CP-SAT (Perron & Furnon, 2023).

3.2. Formal Model

Let $x_i \in \{0, 1\}$ indicate whether candidate i is selected.

$$\max \sum_i x_i \left(w_1 \cdot \text{quality}_i + w_2 \cdot \text{confidence}_i + w_4 \cdot \text{consistency}_i - w_3 \cdot \text{cost}_i \right) \tag{1}$$

$$\text{s.t.} \quad \sum_i x_i = 1 \tag{2}$$

$$\text{quality}_i \geq \text{min_quality} \quad \text{(optional)} \tag{3}$$

$$x_i \in \{0, 1\} \tag{4}$$

$\text{quality}_i = \text{mean}_j\, W[i, j]$ is the Copeland-style aggregate: the mean weight by which candidate i beats every other candidate. Candidates failing the validity predicate are excluded by a hard gate unless every candidate fails. The default weights $(w_1, w_2, w_4, w_3) = (1.0, 0.2, 0.3, 0.01)$ were set by inspection before any ground-truth run and not tuned. Note the units: quality_i and confidence_i lie in $[0, 1]$, whereas cost_i is a word count, so $w_3 = 0.01$ charges one full quality unit per hundred words of difference between candidates. Section 6.5 shows what this does on reasoning-length outputs.

Replay. Each result row stores the candidate texts, the judge's confidence scores and the full pairwise matrix. The selection step can therefore be rerun exactly under any weights without API calls (Section 6.5); the shipped objective reproduces 729 of 729 stored winners.

Worked example. One real open-ended run ($N = 5$) is decomposed term by term in the Online Supplement (Table S9, Figure S1).

3.3. Exact Kemeny–Young Baseline

Selecting a winner by quality_i does not require the full ranking, but the Kemeny–Young rule (Kemeny, 1959; Young & Levenglick, 1978) produces one, and computing it exactly gives a reference for the Copeland approximation. For distinct $i \neq j$ let $y_{ij} \in \{0, 1\}$ indicate that i is ranked above j. The exact ranking solves

$$\max \sum_{i \neq j} W[i, j]\, y_{ij} \tag{5}$$

$$\text{s.t.} \quad y_{ij} + y_{ji} = 1 \quad \forall\, i \neq j \tag{6}$$

$$y_{ij} + y_{jk} + y_{ki} \leq 2 \quad \forall \text{ distinct } i, j, k \tag{7}$$

$$y_{ij} \in \{0, 1\} \tag{8}$$

Constraint (6) forces a total order on every pair; (7) eliminates 3-cycles, which is necessary and sufficient for a linear order. Objective (5) maximizes agreement with the judge's pairwise weights, equivalently minimizing total weighted pairwise disagreement, the quantity Proposition 3.1 bounds. Since $y_{ji} = 1 - y_{ij}$, the implementation (`or_select/kemeny.py`) allocates one binary variable per unordered pair ($\binom{N}{2}$ variables, $O(N^3)$ transitivity constraints). The problem is NP-hard in general (Bartholdi et al., 1989); at $N \leq 6$ CP-SAT solves it exactly.

The reference solver runs with a fixed budget of CP-SAT conflicts rather than wall-clock time, so that "same inputs ⇒ same ranking" holds even when the budget is exhausted; it then returns the best ranking found with an explicit non-optimality flag.

3.4. Approximation Guarantee

Ranking by quality$_i$ is exactly the weighted-out-degree ("weighted number of wins") rule studied by Coppersmith et al. (2010).

Proposition 3.1 (Coppersmith et al., 2010)**.** *Let $W[i,j] + W[j,i] = 1$ for all pairs $i \neq j$. Then ranking candidates by non-increasing quality$_i$ = mean$_j$ $W[i,j]$ is a 5-approximation to the exact Kemeny-optimal ranking, measured by total weighted pairwise disagreement, and the analysis is tight.*

The probability constraint holds by construction ($W[j,i] := 1 - W[i,j]$), so the proposition applies without further assumptions. It bounds the disagreement of the *full ranking*; it does not bound whether the top-ranked candidate equals the exact Kemeny winner, and the shipped objective adds confidence, consistency and cost terms to quality$_i$. We therefore measure the top-1 agreement empirically (Section 6.8). Given fixed inputs the solve is exact: CP-SAT with a single search worker returns the same optimum

regardless of machine or call order. This is true by construction; what we measure is whether the *inputs* to the solve are stable and informative.

4. When can a judge beat voting? Pairwise-accuracy thresholds

The approximation guarantee of Section 3.4 controls the cost of the *full ranking* given the pairwise matrix; it says nothing about whether the top-ranked candidate is correct. The relevant quantity is the accuracy of the pairwise signal itself. We give a threshold for the setting of our homogeneous tracks.

Model. Three candidates are each correct independently with probability p; wrong answers are pairwise distinct, so plurality voting selects a correct answer exactly when at least two candidates are correct (and, with one correct candidate, breaks a three-way tie uniformly). A judge compares each of the three pairs once. On a pair with exactly one correct candidate it prefers the correct one with probability q; on any other pair it chooses uniformly. Selection is the Copeland winner of the three comparisons with ties broken uniformly. This is the quality-only case ($w_2 = w_3 = w_4 = 0$) and therefore the most favourable to the judge.

Theorem 4.1 (Pairwise-accuracy threshold)**.** *Under the model above, the accuracy of the Copeland selector minus the accuracy of plurality voting is*

$$\Delta(p, q) \;=\; p(1-p)\Big[(1-p)(2q-1)(q+1) \;-\; p\,(3-2q)(1-q)\Big].$$

Hence, for $p \in (0, 1)$, the judge-based selector is strictly more accurate than voting if and only if $\frac{p}{1-p} < \frac{(2q-1)(q+1)}{(3-2q)(1-q)}$. In particular, (i) a judge at chance ($q = \frac{1}{2}$) selects with accuracy exactly p and loses to voting by $p^2(1-p)$; (ii) the smallest winning $q^*(p)$ increases strictly from $\frac{1}{2}$ (as $p \to 0$) to 1 (as $p \to 1$).

Table 1 lists $q^*(p)$. The message is that the better the generators already are, the more accurate the judge must be before it adds anything: at $p = 0.8$ the judge must prefer the correct answer in more than 80% of mixed pairs. A

judge helps most where voting is weakest, on weak generators or when correct answers are rare, which is also where a judge is hardest to build. The proof is in Appendix A; the closed form is checked against exhaustive enumeration of the comparison outcomes (`scripts/judge_threshold_theory.py`).

Table 1: Smallest pairwise judge accuracy q^* at which the Copeland selector ties plurality voting, as a function of the per-candidate accuracy p (Theorem 4.1, $N = 3$).

p	$q^*(p)$
0.2	0.565
0.4	0.631
0.5	0.667
0.6	0.706
0.7	0.750
0.8	0.804
0.9	0.875

Lemma 4.1 (Cost dominance)**.** *Let $S_i = w_1 Q_i + w_2 C_i + w_4 K_i - w_3 c_i$ be the selection score of candidate i. If candidate i° has the smallest cost and, for every j, $w_3(c_j - c_{i^\circ}) > w_1(Q_j - Q_{i^\circ})^+ + w_2(C_j - C_{i^\circ})^+ + w_4(K_j - K_{i^\circ})^+$, then every optimum of the selection program contains i° for any judge outputs Q, C, K satisfying those bounds; the judge cannot change the winner.*

Lemma 4.1 is elementary (compare S_{i° with S_j term by term) but it explains an empirical fact: with the shipped weights the cost term dominates the replayed rows (Section 6.5), so the selector is in effect "choose the shortest candidate" and its accuracy equals that of the shortest-candidate rule (Table S6).

Beyond $N = 3$, independent errors and a uniform judge. Three relaxations matter in practice. (a) *Any N.* Let the judge compare every pair once and select the Copeland winner of the N candidates. There is no closed form, but the accuracy is a finite sum over the $2^{N(N-1)/2}$ comparison outcomes, which we enumerate exactly for $N \leq 7$. (b) *A pair-dependent judge.* Let q_{ij} be the probability that the judge prefers the correct candidate i to the wrong

candidate j. (c) *Correlated wrong answers.* A wrong candidate takes one shared "attractor" answer with probability ρ and otherwise a distinct answer; $\rho = 0$ is the model above and $\rho = 1$ makes all wrong answers coincide, as in four-option multiple choice. The judge is unaffected by ρ; plurality is hurt by it.

Results of the extensions. The Online Supplement (Section S1) gives the proofs and tables. Computed exactly for $N = 3, \ldots, 7$ (Table S1), the threshold with distinct wrong answers *rises* with N: at $p = 0.8$ it goes from 0.804 to 0.935, and a large-N lower bound (Proposition S1) shows that this is not an artefact of small N. With correlated wrong answers ($\rho = 0.5$) voting is weaker and the threshold is lower, about 0.82 at $p = 0.8$ for $N \geq 4$. A pair-dependent judge is at least as accurate as voting when $q_{ij} \geq q^*$ on every mixed pair and at most as accurate when $q_{ij} \leq q^*$ (Corollary S1, a monotone-coupling argument). The realistic threshold is therefore far from $\frac{1}{2}$. For GPQA Diamond, whose wrong answers are heavily correlated, it falls from 0.787 to 0.714 (Table 8); the measured $\hat{q} = 0.536$ remains far below it. The exact enumeration agrees with an independent Monte Carlo simulation, and Corollary S1 held in 30 of 30 random pair-dependent instances.

For large N the picture is a regime split rather than a threshold (Proposition S2): a better-than-chance judge can rescue a pool in which the single most popular answer is wrong, where plurality voting fails with certainty, but it cannot be expected to improve a pool in which the right answer already leads. This is the pairwise-judge analogue of the dichotomy established for reward-model best-of-N and majority voting by Di et al. (2025); we claim consistency with it, not novelty. Our benchmark pools are in the right-majority regime (the shipped Nemotron generator is correct on 77–99% of candidates, floor column of Table 5), which is why the judge cannot help.

5. Experimental Protocol

5.1. Setup

Unless noted otherwise, generation, scoring and judging use one free-tier hosted model, NVIDIA Nemotron-3.5-Lightning (`nvidia/nemotron-3.5-lightning:free` on OpenRouter). Judge and confidence calls run with reasoning disabled and a 256-token cap; a cut-off reply raises an error rather than being parsed. Generation is capped at 32,768 output tokens; a cut-off draw is resampled with fresh seeds, and a problem whose slot never completes is recorded as ungraded. Ungraded problems are retried in a flagged second pass at 65,536 tokens (clamped to each model's ceiling); we report accuracy with and without the rescued problems (Table S8).

Heterogeneous generation. $N = 6$ candidates are round-robined over three vendors' models: Nemotron-3.5-Lightning (NVIDIA), Laguna-S 2.1 (Poolside) and North-Mini-Code (Cohere), two candidates per model, with the judge held fixed on Nemotron. Laguna-S 2.1 was withdrawn by the provider while the study ran. Problems graded before that keep their Laguna candidates; for problems generated afterwards its slot is filled by Xing4.0-29B (Xingchen; 256 candidates in total). The tables report how many graded problems still contain Laguna candidates (Table S7). One early GPQA problem also keeps two candidates from LFM-2.5 (2.6B), the first pool's third model.

Homogeneous generation. $N = 3$ candidates from Nemotron with distinct seeds.

5.2. Baselines

Self-consistency: plurality vote over extracted final answers (Wang et al., 2023b), with ties broken by candidate order. **Greedy single-sample**: the first of the N candidates, i.e. no ensembling. **Exact Kemeny–Young**: the winner of the exact ranking of Section 3.3, computed on the same pairwise matrix. Section 6.6 adds further selectors: shortest and longest candidate, minimum-Bayes-risk selection with token-set Jaccard similarity, pointwise confidence argmax, a confidence-weighted vote, a knockout tournament over

the pairwise matrix, and a Bradley–Terry fit (Borda over the pairwise matrix coincides with the Copeland quality). Reward-model or process-reward best-of-N (Cobbe et al., 2021; Lightman et al., 2023) and universal self-consistency (Chen et al., 2023) need a trained model or further live calls and were not run (Section 7.3).

5.3. Judge validation: RewardBench

Following Landesberg's caution that judge calibration does not predict selection accuracy, we first validate the judge in isolation on RewardBench (Lambert et al., 2025): for each (prompt, chosen, rejected) triple, the identical forward+backward judge code path used in the pipeline must prefer `chosen`. The sample is $N = 161$, the first 7 rows of each of 23 subsets (equal per-subset weighting, matching RewardBench's scoring), pre-registered before results were seen. The pre-registration treated accuracy in the range reported for capable general-purpose judges (roughly 70–90%) as support for trusting the judge's quality signal.

5.4. Evaluation tracks

B – GSM8K (Cobbe et al., 2021): 40 test problems, $N = 3$ homogeneous and $N = 6$ heterogeneous; the answer is the last number in the response, because reasoning models restate numbers from the problem before the final answer. **C – MATH Level 5**: the 134 Level-5 problems of MATH-500 (Hendrycks et al., 2021; Lightman et al., 2023), exact-match on the `\boxed{}` answer. **D – AIME 2026**: 30 problems (Balunović et al., 2025). **E – GPQA Diamond**: 198 graduate-level multiple-choice questions (Rein et al., 2024), exact-letter grading. **F – AIME 2025**: the 30 problems of AIME I and II 2025 (Balunović et al., 2025), homogeneous generation only. It predates the generator, so contamination is possible and the track is a robustness check, not an uncontaminated one. **G – MMLU-Pro** (Wang et al., 2024): a deterministic 100-question sample of the test split (five consecutive questions at each of 20 evenly spaced offsets, independent of any model output; 12 of the 14 subjects occur), up to ten lettered options,

exact-letter grading, homogeneous generation only. Tracks C–G are chosen because they are not saturated for the model used (Kim et al., 2025; Ding, 2026). Track A (three open-ended queries without ground truth) is reported only as a pilot illustration (Section S5).

5.5. Metrics

Coverage. Every table reports graded/total, where total is the benchmark size, never the number of rows attempted; ungraded problems are not silently dropped (Table S7). *Paired tests.* For two selectors on the same graded problems we count discordant problems and report the exact two-sided McNemar (binomial) test, the test we pre-registered (the statistical treatment of paired model comparisons follows Miller, 2024); pooled tests sum discordant counts over tracks. No multiplicity correction is applied: the pooled homogeneous and heterogeneous tests are the primary comparisons, and per-track p-values are descriptive. *Headroom and recovery.* For each problem we score every candidate. The *floor* is the mean candidate accuracy (a random pick), the *oracle* is the fraction of problems with at least one correct candidate, and a selector's *recovery* is $(\text{acc} - \text{floor})/(\text{oracle} - \text{floor})$ (cf. Landesberg, 2026). Recovery shows how much of what could be gained a selector captures; negative values mean it does worse than a random pick. *Offline ablations* (Section 6.5) replay the selection step on stored judge outputs; they are post hoc and exploratory.

6. Results

6.1. Judge validation: RewardBench

Overall accuracy is 60.5% (mean of category means, $N = 161$, micro-average 60.9%, 95% Wilson interval for the micro-average [53.2%, 68.1%]), below the range the pre-registration treated as sufficient to trust the quality signal. Table 2 gives the breakdown. Reasoning pairs, the category most relevant to Tracks B–G, reach 73.5% (36/49); Chat Hard, which is adversarial, is at 42.9%, below the 50% chance level. When the judge is rotated

across the three heterogeneous-pool models instead of being held fixed, overall accuracy is 48.5% (N=161), indistinguishable from guessing; the pipeline therefore keeps the judge fixed.

A length diagnostic on the same rows splits by whether the human-preferred response is the longer one: accuracy is 68.4% (54/79, 95% CI [57.5%, 77.6%]) when it is, and 53.7% (44/82, [42.9%, 64.0%]) when it is shorter; the intervals overlap, so this is a direction to watch rather than a proven bias.

An earlier pilot, run on 2026-08-22 with a larger model (Nemotron-3-Ultra, since withdrawn by the provider) and a reasoning-enabled judge call, reported a higher figure (90.1%). The judge call was changed to its reasoning-disabled, 256-token form on 2026-10-07 because the reasoning-enabled calls were frequently cut off at the token cap (six of ten in a spot check). That pilot used a different model and judge configuration from every result below, and we do not rely on it.

Table 2: `judge_fn` accuracy on RewardBench, by category ($N = 161$).

Category	Rows	Accuracy
Reasoning (math-prm, 6 `hep-*` code subsets)	49	73.5%
Chat	35	68.6%
Chat Hard (adversarial LLMBar subsets)	42	42.9%
Safety	35	57.1%

6.2. Accuracy against the baselines

Table 3 and Figure 2 give accuracy on graded problems. With homogeneous generation (rows B–G) the three non-exact selectors are within a few problems of each other on every track. On GSM8K all candidates agree on all 40 problems, so no selector can differ; on MATH Level 5 self-consistency is ahead of OR-Select by two problems; on AIME 2026 OR-Select is ahead by one; on AIME 2025 self-consistency is ahead by two; on GPQA Diamond and MMLU-Pro they tie. Table 4 makes the comparison paired: pooled over the

six homogeneous tracks, OR-Select is right where self-consistency is wrong on 10 problems and wrong where it is right on 13 ($p = 0.678$, $n = 532$). There is no evidence of an advantage or a disadvantage.

With heterogeneous generation (rows B′–E′) OR-Select falls behind on three of the four tracks: on MATH Level 5 it is right where self-consistency is wrong on 0 problems and wrong where it is right on 15 (79.1% versus 90.3%), on AIME 2026 on 0 versus 9, on GSM8K on 0 versus 2, and pooled over the four heterogeneous tracks on 10 versus 32 problems ($n = 402$, $p < 0.001$). GPQA Diamond is the exception: there OR-Select is nominally ahead of self-consistency (right where it is wrong on 10 problems, wrong where it is right on 6; exact $p = 0.454$), so the pooled harm is carried by MATH, AIME and GSM8K. All heterogeneous tracks are fully graded (Table S7).

Table 3: Accuracy (%) on graded problems, by track and selector. '–': no Kemeny–Young comparator (GSM8K). Bold: best of the three practical selectors when they differ. Primed rows: heterogeneous generation.

Track	Graded/Total	Greedy	Self-cons.	OR-Select	Kemeny–Young
B — GSM8K, homog.	40/40	100.0	100.0	100.0	–
B′ — GSM8K, heterog.	40/40	**100.0**	**100.0**	95.0	–
C — MATH L5, homog.	134/134	89.6	**91.0**	89.6	90.3
C′ — MATH L5, heterog.	134/134	88.1	**90.3**	79.1	79.9
D — AIME 2026, homog.	30/30	80.0	86.7	**90.0**	83.3
D′ — AIME 2026, heterog.	30/30	86.7	**93.3**	63.3	90.0
E — GPQA Diamond, homog.	198/198	75.3	**78.8**	**78.8**	78.3
E′ — GPQA Diamond, heterog.	198/198	74.7	77.3	**79.3**	72.7
F — AIME 2025, homog.	30/30	80.0	**83.3**	76.7	86.7
G — MMLU-Pro, homog.	100/100	85.0	**86.0**	**86.0**	83.0

6.3. Headroom and recovery

Accuracy alone hides how much a selector could have gained (Table 5). On GSM8K there is none: floor and oracle coincide. On GPQA Diamond with homogeneous generation the oracle is 87.4% against a floor of 77.1%, so a perfect selector would add about ten points; both self-consistency and OR-Select capture 16% of that headroom. On heterogeneous MATH Level 5 and AIME 2026, self-consistency recovers most of the headroom while OR-Select

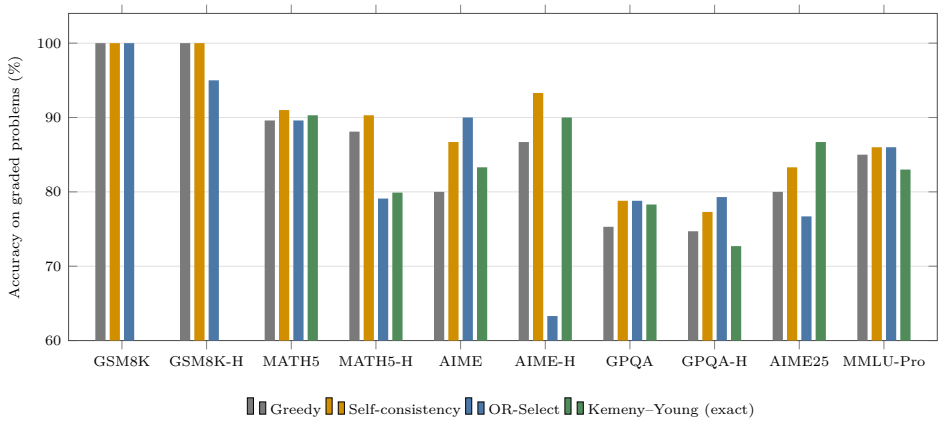


Figure 2: Accuracy on graded problems by track and selector, from Table 3. "-H": heterogeneous generation. Coverage differs across bars (Table S7).

Table 4: Paired comparison of OR-Select with each reference selector. Cells give (problems where OR-Select is right and the reference wrong)/(the reverse) and the exact two-sided McNemar p-value.

Track	vs. SC	p	vs. greedy	p	vs. Kemeny–Young	p
B — GSM8K, homog.	0/0	–	0/0	–	–	–
B′ — GSM8K, heterog.	0/2	0.500	0/2	0.500	–	–
C — MATH L5, homog.	0/2	0.500	1/1	1.000	1/2	1.000
C′ — MATH L5, heterog.	0/15	<0.001	4/16	0.012	7/8	1.000
D — AIME 2026, homog.	1/0	1.000	3/0	0.250	2/0	0.500
D′ — AIME 2026, heterog.	0/9	0.004	0/7	0.016	1/9	0.021
E — GPQA Diamond, homog.	6/6	1.000	13/6	0.167	9/8	1.000
E′ — GPQA Diamond, heterog.	10/6	0.454	20/11	0.150	24/11	0.041
F — AIME 2025, homog.	1/3	0.625	1/2	1.000	1/4	0.375
G — MMLU-Pro, homog.	2/2	1.000	3/2	1.000	5/2	0.453

scores *below* the random-pick floor. The selector is not merely failing to help; in those heterogeneous pools it is anti-informative. Heterogeneous GPQA Diamond is the exception, where OR-Select recovers 35% of the headroom against 26% for self-consistency, a difference that is not significant (Table 4). On the two added homogeneous tracks the two selectors recover the same share on MMLU-Pro (14%), and OR-Select falls below the floor on AIME 2025.

Table 5: Headroom and recovery on graded problems. Floor: mean candidate accuracy. Oracle: some candidate correct. Recovery (%): (acc − floor)/(oracle − floor); negative means below a random pick.

Track	Graded	Floor	SC	OR	Oracle	Rec. SC	Rec. OR
B — GSM8K, homog.	40	99.2	100.0	100.0	100.0	100	100
B′ — GSM8K, heterog.	40	97.5	100.0	95.0	100.0	100	-100
C — MATH L5, homog.	134	89.8	91.0	89.6	91.8	62	-12
C′ — MATH L5, heterog.	134	81.8	90.3	79.1	91.8	85	-27
D — AIME 2026, homog.	30	85.6	86.7	90.0	90.0	25	100
D′ — AIME 2026, heterog.	30	76.7	93.3	63.3	96.7	83	-67
E — GPQA Diamond, homog.	198	77.1	78.8	78.8	87.4	16	16
E′ — GPQA Diamond, heterog.	198	71.8	77.3	79.3	92.9	26	35
F — AIME 2025, homog.	30	81.1	83.3	76.7	93.3	18	-36
G — MMLU-Pro, homog.	100	85.0	86.0	86.0	92.0	14	14

6.4. Why heterogeneous selection fails: the judge prefers one generator

Across the heterogeneous problems we know which model produced every candidate and which candidate OR-Select picked (Table 6). With a uniform pool each model would be picked a third of the time. The judge picks the Cohere candidate in 75.6% of problems although Cohere's candidates (83.9% correct) are as accurate as Nemotron's (82.5%). It picks Nemotron, the judge's own model, in only 6.1% of problems, so the self-preference reported in the literature (Panickssery et al., 2024; Wataoka et al., 2024) is not the explanation here. The picked Cohere candidates are right 79.6% of the time, below the 83.9% of Cohere's candidates in general: the preference selects a style, not correctness. The weakest model, Xing4.0-29B (56.2% correct),

is picked in 2.6% of problems with 44.4% accuracy, and Laguna-S (73.4% correct) in 15.4% with 67.9%. The winner is the longest candidate in only 1% of heterogeneous problems (chance 17%) and 6% of homogeneous problems (chance 33%); on average it has 0.38 (hetero.) and 0.70 (homog.) times the mean candidate length.

Table 6: Heterogeneous pools, pooled over the four tracks (402 graded problems). Candidates: all candidates generated by the model and their accuracy (%). Picks: how often OR-Select selected that model's candidate, with share, and the accuracy of those picks (%).

Model	Candidates	Cand. acc.	OR-Select picks	Pick acc.
Nemotron-3.5 (judge model)	804	82.5	21 (6.1%)	90.5
North-Mini-Code	802	83.9	260 (75.6%)	79.6
Laguna-S 2.1	548	73.4	53 (15.4%)	67.9
Xing4.0-29B	256	56.2	9 (2.6%)	44.4
LFM-2.5 2.6B	2	50.0	1 (0.3%)	100.0

6.5. Diagnosing the objective: scale and the judge's own signal

Because every stored row carries the judge's outputs, we replay the selection step under other weights at no API cost (729/729 stored winners are reproduced by the shipped objective, so the replay is faithful). Two facts follow.

The cost term dominates. Within a problem, the spread of the cost term $w_3 \cdot \text{cost}_i$ across candidates has median 0.88 against 0.20 for the quality term, and exceeds it in 653 of 729 replayed problems. On reasoning-length outputs the default objective is, to a first approximation, "pick the shortest valid candidate".

Removing it is not enough in heterogeneous pools. Table 7 replays four variants: the default; no cost term; Copeland quality alone; and plurality vote with Copeland quality as the tie-break. With homogeneous generation all variants are within about one point of self-consistency (the default and no-cost variants win 9 and 11 problems against self-consistency's 10 and 12; $n = 332$). With heterogeneous generation the no-cost and quality-only variants

recover only part of the gap (78.6% and 78.1% against self-consistency's 84.9%, $n = 397$; exact McNemar p <0.001 and <0.001 respectively), and even using the judge only to break ties in a plurality vote does not beat plain voting (wins/losses 1/7). The judge's pairwise signal thus carries no accuracy information beyond voting here, and it is harmful when candidates come from different models. These ablations are post hoc and exploratory; the primary comparison is the pre-registered one in Section 6.2.

Table 7: Offline replay of the selection step on stored judge outputs (accuracy %, problems whose judge outputs were stored). SC: stored self-consistency result on the same problems.

Track	Replayable	SC	Default	No cost	Quality only	SC+quality
B′ GSM8K, heterog.	38/40	100.0	94.7	100.0	100.0	100.0
C MATH L5, homog.	25/134	84.0	84.0	84.0	84.0	84.0
C′ MATH L5, heterog.	133/134	90.2	78.9	82.7	82.7	87.2
D AIME 2026, homog.	9/30	66.7	66.7	66.7	66.7	66.7
D′ AIME 2026, heterog.	29/30	93.1	62.1	75.9	75.9	96.6
E GPQA Diamond, homog.	168/198	77.4	78.0	78.0	78.0	76.2
E′ GPQA Diamond, heterog.	197/198	77.2	79.2	72.1	71.1	75.6
F AIME 2025, homog.	30/30	83.3	76.7	83.3	83.3	83.3
G MMLU-Pro, homog.	100/100	86.0	86.0	84.0	84.0	86.0

6.6. Additional selectors

We also compared OR-Select's competitors beyond Section 6.2: judge-free selectors (shortest and longest candidate, minimum-Bayes-risk selection with token-set Jaccard similarity), run on every graded problem, and judge-based selectors replayed on the problems whose judge outputs were stored (pointwise confidence argmax, a confidence-weighted vote, a knockout tournament over the pairwise matrix, and a Bradley–Terry fit). Each is paired with self-consistency on the same problems (Table S6 in the Online Supplement). No selector beats plurality voting in either condition. The judge-free minimum-Bayes-risk selector and the confidence-weighted vote, itself a variant of voting, stay within a few problems of voting; the selectors that act on the judge's pairwise or pointwise output choose worse than voting in heterogeneous pools, as OR-Select does, and the trivial length rules are worse still.

The common thread is the one of Section 6.5: whatever signal the judge adds is not about correctness.

6.7. Measured pairwise accuracy against the thresholds

We measure $\hat{p}$ (share of correct candidates) and $\hat{q}$ (the mean stored pairwise probability assigned to the correct candidate, over ordered pairs with exactly one correct candidate) on the stored judge outputs, and compare $\hat{q}$ with the exact threshold q^* of Section 4, first with distinct wrong answers (q_0^*) and then with the measured wrong-answer correlation $\hat{\rho}$ (Table 8). Only tracks with at least 20 mixed ordered pairs are shown: for homogeneous generation that is GPQA Diamond and MMLU-Pro (28 mixed pairs), because the other homogeneous tracks stored too few judgments containing a mixed pair.

The measured $\hat{q}$ is far below every threshold. For GPQA Diamond the judge prefers the correct candidate in only 53.6% of mixed comparisons against a required 78.7% (71.4% once the very high wrong-answer correlation, $\hat{\rho} = 0.88$, is allowed for); for MMLU-Pro the figures are 44.3% against 83.6% (78.6%). On the heterogeneous tracks $\hat{q}$ lies between 0.49 and 0.56 on every track with at least 70 mixed pairs, which is at or near chance; GSM8K (0.66, 24 mixed pairs) has only three wrong pairs, so ρ cannot be estimated, its threshold interval spans every possible ρ, and $\hat{q}$ is still below the lower end. This is the pairwise-level counterpart of the selection-level results above: the judge's signal about correctness is too weak for any selector built on it to beat voting. The model is stylized (independent candidates, one wrong-answer structure, one accuracy for all pairs), so we use it for the threshold comparison, not to predict accuracies. The replayable rows are the ones that stored judge outputs, not a random sample of problems (Section 7.3).

6.8. Exact Kemeny–Young reference

Across the 854 graded problems that have an exact Kemeny–Young solution, the shipped selector and the exact winner give different answers on 139. The selector alone is right on 50 and the exact winner alone on 44 problems,

Table 8: Thresholds with measured wrong-answer correlation. "Wrong pairs": pairs of wrong candidates of the same problem with extracted answers; $\hat{\rho}$: square root of their agreement rate, with a 95% problem-level bootstrap interval; q_0^*: exact threshold at $\rho = 0$; $q_{\hat{\rho}}^*$: exact threshold at $\hat{\rho}$, with the values at the ends of the $\hat{\rho}$ interval; $\hat{q}$ is computed over the mixed ordered pairs. ‡: fewer than 30 wrong pairs, so ρ is not estimated and the interval spans every possible value, from $\rho = 1$ to $\rho = 0$. Suffix H: heterogeneous ($N = 6$). $\hat{q}$ is below the lower end of every threshold interval.

Track	N	$\hat{p}$	Wrong pairs	$\hat{\rho}$ [95% CI]	Mixed pairs	$\hat{q}$	q_0^*	$q_{\hat{\rho}}^*$
GPQA Diamond	3	0.771	94	0.88 [0.82, 0.93]	72	0.536	0.787	0.714 [0.706, 0.724]
MMLU-Pro	3	0.850	31	0.84 [0.66, 0.96]	28	0.443	0.836	0.786 [0.772, 0.805]
GSM8K-H	6	0.975	3	n/a	24	0.660	0.995	[0.904, 0.995]‡
MATH L5-H	6	0.818	219	0.66 [0.50, 0.80]	292	0.536	0.941	0.789 [0.755, 0.828]
AIME 2026-H	6	0.767	32	0.31 [0.00, 0.47]	146	0.560	0.923	0.857 [0.809, 0.923]
GPQA Diamond-H	6	0.718	530	0.82 [0.77, 0.87]	615	0.489	0.905	0.682 [0.667, 0.697]

so the exact solution is not more accurate either (Table 4, last columns). This is the empirical counterpart of Proposition 3.1: the full-ranking guarantee does not control the top-1 winner, and in this setting neither aggregation rule helps because both consume the same weak pairwise signal.

6.9. Coverage and the higher-cap second pass

Table S7 reports coverage, which would also show problems that never produced a graded answer (truncated at the token cap after resampling, other failures, or not yet attempted); in the final data there are none, and every problem of every track is graded. The second pass at 65,536 tokens rescued the problems in Table S8. These are the problems with the longest reasoning, and they are harder: including them lowers accuracy by between 0.3 and 15 points for either selector (the largest drops are on the 30-problem AIME tracks), without changing the direction of any comparison above.

6.10. Pilot illustration: open-ended queries

Three open-ended queries ($N = 5$ each, no ground truth) were run in an earlier pilot (Online Supplement, Section S5). None of the 15 candidates was byte-identical to another, so self-consistency fell back to an arbitrary tie-break on every query, and OR-Select differed from both baselines on all

three. With three queries and no ground truth this is an illustration, not evidence about quality.

7. Discussion

7.1. What the audit shows

The method does what it is designed to do: selection is exactly reproducible and replayable, with an exact Kemeny–Young reference. It does not improve on majority vote, for four measured reasons. First, the judge is only moderately reliable and, at 42.9% on adversarial pairs, poor where candidates are close. Second, the headroom is small on the saturated benchmarks (floor $\approx$ oracle on GSM8K, within two points on MATH Level 5). Third, where headroom exists (homogeneous GPQA Diamond) the judge's pairwise signal recovers no more of it than a vote does; on heterogeneous GPQA it recovers somewhat more, but not significantly so. Fourth, in heterogeneous pools the judge's preference for one generator and the objective's length-scale mismatch turn a neutral selector into a harmful one. This adds a within-pool, ground-truth measurement to the literature on correlated errors and on the weak link between judge metrics and selection accuracy (Kohli, 2026; Kim et al., 2025; Landesberg, 2026).

Why the judge cannot help, quantitatively. The threshold analysis of Section 4 turns the first three factors into one number. When generators are already accurate ($\hat{p} \approx 0.8$) a pairwise judge must prefer the correct answer in roughly 70–95% of mixed pairs, depending on N and on how strongly wrong answers collide, before Copeland aggregation can match plurality voting; the measured value is 0.44–0.56 (GSM8K, with three wrong pairs, 0.66). A judge can pay off where the most popular answer is wrong (Proposition S2), which none of our benchmark pools is.

7.2. A non-determinism observation

In an earlier pilot on a different provider and model, three byte-identical candidates received judge scores of 0.375, 0.50 and 0.625, although identical

text must score 0.5 by construction: direct evidence that the provider's `seed` parameter was not reliably honored (details and the extraction fallbacks in the Online Supplement, Section S6), in line with systematic run-to-run variation reported for nominally deterministic settings (Atil et al., 2024). The exact-match short-circuit removes the failure mode for identical text but not for different texts that receive unstable scores, which remains open.

7.3. Limitations

One judge model. Generation, scoring and judging use one free-tier model, which the RewardBench result shows to be only moderately reliable; a stronger or more diverse judge may change the conclusions, and we cannot say how. Only generator diversity was tested, and rotating weak judges is not a remedy.

Mixed pool. Heterogeneous problems graded before Laguna-S 2.1 was withdrawn keep its candidates (Table S7), and one GPQA problem retains LFM-2.5 candidates, so the heterogeneous rows describe a mixed pool, not one fixed pool. AIME 2025 and MMLU-Pro are homogeneous only; AIME 2025 predates the generator and may be contaminated, and the MMLU-Pro sample covers 12 of 14 subjects.

Power and post hoc analyses. Candidate sets are small ($N = 3$–6) and the homogeneous comparison has few discordant problems, so large effects can be excluded but not small ones. The weight ablations, selection-share and length analyses, the AIME 2025 and MMLU-Pro tracks and the six-track pooled test were not pre-registered; they confirm the earlier direction but are not independent pre-registrations. The default weights were fixed before the ground-truth runs and not tuned; we report the shipped objective as the primary result.

Theory scope. The thresholds assume independent candidates, a symmetric judge and either one shared wrong answer or wrong answers uniform over D values; real wrong answers are skewed and problem-specific. Exact values stop at $N = 7$ and the quality-only model ignores the cost term. The measured $\hat{q}$ comes from rows that stored judge outputs, not a random

sample, and contains few mixed pairs on most homogeneous tracks.

Scope. We did not run reward-model or process-reward best-of-N, which need a trained verifier, nor universal self-consistency. Tracks are mathematics and science questions with checkable answers; open-ended quality, where a judge is unavoidable, is covered only by a three-query pilot.

8. Conclusion

We formulated best-of-N answer selection as an exactly reproducible integer program over a pairwise matrix and a deterministic self-consistency term, inherited a 5-approximation guarantee against the NP-hard Kemeny-optimal ranking, and built an audit around it: exact Kemeny–Young reference, paired exact tests, headroom and recovery, and replayable judge outputs. On six ground-truth benchmarks the audit finds no accuracy benefit of judge-based selection over majority vote with a single generator, and a significant harm pooled over the heterogeneous tracks (carried by MATH, AIME and GSM8K; GPQA shows none), traced to a length-scale mismatch in the objective and a generator-specific judge preference rather than to nondeterminism. A threshold analysis shows why: the judge's measured pairwise accuracy sits far below the accuracy the aggregation would need. The practical lessons are to validate the judge on the configuration actually shipped, to report recovery against a floor and an oracle, and to keep judge outputs so that a selector can be audited without new API calls. Whether a stronger or more diverse judge makes selection pay off is the open question this protocol is built to answer.

Data and code availability

Code, stored run files (candidates, judge outputs and graded results) for every track except GPQA Diamond, generated tables and the pre-registration documents are archived on Zenodo (Dressino, 2026) and developed at `https://github.com/DressPD/or-select`. The GPQA Diamond files embed the benchmark's password-gated question text, so they are not republished; they

are regenerated by the released scripts, or available from the author on request. All numbers in Section 6 are regenerated from the stored outputs by the scripts named at the top of the source (the GPQA rows need the local GPQA files).

CRediT authorship contribution statement

Aris Dressino: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author used Claude (Anthropic; Claude Code with Claude Sonnet 5.5) to write and refactor analysis code, to draft and edit manuscript text, to check bibliographic records against Crossref and arXiv, and to derive and numerically verify the results in Section 4 and the appendices. The language models named in the experiments (served through OpenRouter) are the objects of study, not writing tools. After using these tools the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Appendix A. Proof of Theorem 4.1

Let $a_k(q)$ be the probability that the Copeland selector returns a correct candidate given that exactly k of the three candidates are correct. Trivially $a_0 = 0$ and $a_3 = 1$.

One correct candidate C, two wrong W_1, W_2. C beats each W_i with probability q and W_1 vs. W_2 is a fair coin. If C wins both (probability q^2) its score is 2 and it is the unique winner. If C wins exactly one (probability $2q(1-q)$), say it beats W_1 and loses to W_2, then W_2 is the unique winner when it also beats W_1 (probability $\frac{1}{2}$); otherwise all three scores equal 1 and C is chosen with probability $\frac{1}{3}$. If C loses both, its score is 0 and it is never chosen. Hence $a_1(q) = q^2 + 2q(1-q) \cdot \frac{1}{2} \cdot \frac{1}{3} = q^2 + \frac{1}{3}q(1-q)$.

Two correct candidates C_1, C_2, one wrong W. C_1 vs. C_2 is a fair coin and each C_i beats W with probability q. If W beats both (probability $(1-q)^2$) it has score 2 and wins. If W beats exactly one, say C_i (probability $2q(1-q)$), then W has score 1 and C_j has score $1 + [C_j \text{ beats } C_i]$: if C_j beats C_i it wins alone; otherwise all three scores are 1 and a correct candidate is chosen with probability $\frac{2}{3}$, giving $\frac{1}{2} + \frac{1}{2} \cdot \frac{2}{3} = \frac{5}{6}$. If W beats neither, its score is 0 and a correct candidate is chosen. Hence $a_2(q) = q^2 + \frac{5}{3}q(1-q)$.

Plurality voting with distinct wrong answers has $v_0 = 0$, $v_1 = \frac{1}{3}$, $v_2 = v_3 = 1$. With $k \sim \mathrm{Bin}(3, p)$,

$$\Delta = 3p(1-p)^2\big(a_1 - \tfrac{1}{3}\big) + 3p^2(1-p)\big(a_2 - 1\big).$$

Now $a_1 - \frac{1}{3} = \frac{1}{3}(2q-1)(q+1)$ and $a_2 - 1 = -\frac{1}{3}(3-2q)(1-q)$, which gives the stated $\Delta(p, q)$. Positivity of Δ is equivalent to the stated inequality because $p(1-p) > 0$. (i) At $q = \frac{1}{2}$ the first term vanishes and $\Delta = -p^2(1-p)$; a chance-level judge picks uniformly, so its accuracy is p and voting has accuracy $p + p^2(1-p)$. (ii) The right-hand side of the inequality, $g(q) = \frac{(2q-1)(q+1)}{(3-2q)(1-q)}$, is continuous and strictly increasing on $(\frac{1}{2}, 1)$ (numerator increasing from 0, denominator decreasing to 0), with $g(\frac{1}{2}) = 0$ and $g(q) \to \infty$ as $q \to 1$. Therefore for each $p/(1-p) \in (0, \infty)$ there is a unique $q^*(p) \in (\frac{1}{2}, 1)$ with $g(q^*) = p/(1-p)$; q^* is strictly increasing in p with $q^* \to \frac{1}{2}$ as $p \to 0$ and $q^* \to 1$ as $p \to 1$. □

References

Ai, R., Pan, Y., Simchi-Levi, D., Tambe, M., & Xu, H. (2025). *Beyond majority voting: LLM aggregation by leveraging higher-order information.* ICML 2026.

Aktaş, D., Lokman, B., İnkaya, T., & Dejaegere, G. (2024). Cluster ensemble selection and consensus clustering: A multi-objective optimization approach. *European Journal of Operational Research*, 314(3), 1065–1077. https://doi.org/10.1016/j.ejor.2023.10.029

Aledo, J. A., Gámez, J. A., & Rosete, A. (2018). Approaching rank aggregation problems by using evolution strategies: The case of the optimal bucket order problem. *European Journal of Operational Research*, 270(3), 982–998. https://doi.org/10.1016/j.ejor.2018.04.031

Amodio, S., D'Ambrosio, A., & Siciliano, R. (2016). Accurate algorithms for identifying the median ranking when dealing with weak and partial rankings under the Kemeny axiomatic approach. *European Journal of Operational Research*, 249(2), 667–676. https://doi.org/10.1016/j.ejor.2015.08.048

Atil, B., Aykent, S., Chittams, A., Fu, L., Passonneau, R. J., Radcliffe, E., Rajagopal, G. R., Sloan, A., Tudrej, T., Ture, F., Wu, Z., Xu, L., & Baldwin, B. (2024). *Non-determinism of "deterministic" LLM settings.*

Azzini, I. & Munda, G. (2020). A new approach for identifying the Kemeny median ranking. *European Journal of Operational Research*, 281(2), 388–401. https://doi.org/10.1016/j.ejor.2019.08.033

Balunović, M., Dekoninck, J., Petrov, I., Jovanović, N., & Vechev, M. (2025). *MathArena: Evaluating LLMs on uncontaminated math competitions.* SRI Lab, ETH Zurich, https://matharena.ai/.

Bartholdi, J. J., Tovey, C. A., & Trick, M. A. (1989). Voting schemes for which it can be difficult to tell who won the election. *Social Choice and Welfare*, 6(2), 157–165. https://doi.org/10.1007/BF00303169

Brown, A. & Reade, J. J. (2019). The wisdom of amateur crowds: Evidence from an online community of sports tipsters. *European Journal of Operational Research*, 272(3), 1073–1081. https://doi.org/10.1016/j.ejor.2018.07.015

Brown, B., Juravsky, J., Ehrlich, R., Clark, R., Le, Q. V., Ré, C., & Mirhoseini, A. (2024). *Large language monkeys: Scaling inference compute with repeated sampling.*

Burka, D., Puppe, C., Szepesvári, L., & Tasnádi, A. (2022). Voting: A machine learning approach. *European Journal of Operational Research*, 299(3), 1003–1017. https://doi.org/10.1016/j.ejor.2021.10.005

Caragiannis, I., Procaccia, A. D., & Shah, N. (2016). When do noisy votes reveal the truth? *ACM Transactions on Economics and Computation*, 4(3), 1–30. https://doi.org/10.1145/2892565

Chen, X., Aksitov, R., Alon, U., Ren, J., Xiao, K., Yin, P., Prakash, S., Sutton, C., Wang, X., & Zhou, D. (2023). *Universal self-consistency for large language model generation.*

Chen, Z., Ji, Z., Mao, Q., Wu, H., Song, J., Cheng, J., Qin, B., Li, Z., Li, J., Sun, K., Wang, Z., Ban, Y., Sun, Z., Ji, X., Sun, H., & Huang, X. (2025). *Scoring, reasoning, and selecting the best! ensembling large language models via a peer-review process.*

Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., Hesse, C., & Schulman, J. (2021). *Training verifiers to solve math word problems.*

Conitzer, V. & Sandholm, T. (2005). Common voting rules as maximum likelihood estimators. *Proceedings of the 21st Conference on Uncertainty in Artificial Intelligence (UAI)*, 145–152.

Coppersmith, D., Fleischer, L., & Rudra, A. (2010). Ordering by weighted number of wins gives a good ranking for weighted tournaments. *ACM Transactions on Algorithms*, 6(3), 55:1–55:13.

De Baets, S. & Harvey, N. (2020). Using judgment to select and adjust forecasts from statistical models. *European Journal of Operational Research*, 284(3), 882–895. https://doi.org/10.1016/j.ejor.2020.01.028

De Bock, K. W., Coussement, K., & Lessmann, S. (2020). Cost-sensitive business failure prediction when misclassification costs are uncertain: A heterogeneous ensemble selection approach. *European Journal of Operational Research*, 285(2), 612–630. https://doi.org/10.1016/j.ejor.2020.01.052

Di, Q., Ji, K., Li, X., Zhao, H., & Gu, Q. (2025). *Best-of-majority: Minimax-optimal strategy for pass@k inference scaling.* ICLR 2026.

Dietrich, F. & List, C. (2004). A model of jury decisions where all jurors have the same evidence. *Synthese*, 142(2), 175–202. https://doi.org/10.1007/s11229-004-1276-z

Ding, K. (2026). *When llms agree, are they right? auditing self-consistency and cross-model agreement as confidence signals.*

Dressino, A. (2026). *OR-Select: code, stored run files and generated tables.* https://doi.org/10.5281/zenodo.24112045. [dataset]

Dwork, C., Kumar, R., Naor, M., & Sivakumar, D. (2001). Rank aggregation methods for the Web. *Proceedings of the 10th International Conference on World Wide Web (WWW)*, 613–622. https://doi.org/10.1145/371920.372165

Gu, J., Jiang, X., Shi, Z., Tan, H., Zhai, X., Xu, C., Li, W., Shen, Y., Ma, S., Liu, H., Wang, S., Zhang, K., Wang, Y., Gao, W., Ni, L., & Guo, J. (2024). *A survey on LLM-as-a-judge.*

Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., & Steinhardt, J. (2021). Measuring mathematical problem solving with the MATH dataset. *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track.*

Hu, Z., Zhang, J., Xiong, Z., Ratner, A., Ding, K., & Krishna, R. (2024). *Towards acyclic preference evaluation of language models via multiple evaluators.*

Jaspersen, J. G. (2022). Convex combinations in judgment aggregation. *European Journal of Operational Research*, 299(2), 780–794. https://doi.org/10.1016/j.ejor.2021.09.050

Jiang, D., Ren, X., & Lin, B. Y. (2023). Llm-blender: Ensembling large language models with pairwise ranking and generative fusion. *Proceedings of the Association for Computational Linguistics (ACL).*

Kemeny, J. G. (1959). Mathematics without numbers. *Daedalus*, 88(4), 577–591.

Kim, D. (2026). *Are diversity metrics measuring diversity? a capability-controlled audit of majority-vote gain in LLM ensembles.*

Kim, E., Garg, A., Peng, K., & Garg, N. (2025). Correlated errors in large language models. *International Conference on Machine Learning (ICML).*

Kohli, G. (2026). *Nine judges, two effective votes: Correlated errors undermine llm evaluation panels.*

Komiyama, J., Oba, D., & Oyamada, M. (2025). *Best-of-∞ – asymptotic performance of test-time LLM ensembling.* ICLR 2026.

Ladha, K. K. (1992). The Condorcet jury theorem, free speech, and correlated votes. *American Journal of Political Science*, 36(3), 617–634. https://doi.org/10.2307/2111584

Lambert, N., Pyatkin, V., Morrison, J., Miranda, L., Lin, B. Y., Chandu, K., Dziri, N., Kumar, S., Zick, T., Choi, Y., Smith, N. A., & Hajishirzi, H. (2025). Rewardbench: Evaluating reward models for language modeling. *Findings of the Association for Computational Linguistics: NAACL 2025.*

Landesberg, E. (2026). *When llm judge scores look good but best-of-n decisions fail.*

Lefort, B., Benhamou, E., Ohana, J.-J., Guez, B., Saltiel, D., & Jacquot, T. (2024). *Examining independence in ensemble sentiment analysis: A study on the limits of large language models using the Condorcet jury theorem.*

Lightman, H., Kosaraju, V., Burda, Y., Edwards, H., Baker, B., Lee, T., Leike, J., Schulman, J., Sutskever, I., & Cobbe, K. (2023). *Let's verify step by step.*

Liu, Y., Zhou, H., Guo, Z., Shareghi, E., Vulić, I., Korhonen, A., & Collier, N. (2024). Aligning with human judgement: The role of pairwise preference in large language model evaluators. *Conference on Language Modeling (COLM).*

Maarouf, A., Feuerriegel, S., & Pröllochs, N. (2025). A fused large language model for predicting startup success. *European Journal of Operational Research*, 322(1), 198–214. https://doi.org/10.1016/j.ejor.2024.09.011

Miller, E. (2024). *Adding error bars to evals: A statistical approach to language model evaluations.*

Panickssery, A., Bowman, S. R., & Feng, S. (2024). *LLM evaluators recognize and favor their own generations.*

Perron, L. & Furnon, V. (2023). *OR-Tools CP-SAT solver.* Google, https://developers.google.com/optimization/.

Rein, D., Hou, B. L., Stickland, A. C., Petty, J., Pang, R. Y., Dirani, J., Michael, J., & Bowman, S. R. (2024). Gpqa: A graduate-level google-proof q&a benchmark. *Conference on Language Modeling (COLM).*

Rico, N., Vela, C. R., & Díaz, I. (2023). Reducing the time required to find the Kemeny ranking by exploiting a necessary condition for being a winner. *European Journal of Operational Research*, 305(3), 1323–1336. https://doi.org/10.1016/j.ejor.2022.07.031

Snell, C., Lee, J., Xu, K., & Kumar, A. (2024). *Scaling LLM test-time compute optimally can be more effective than scaling model parameters.*

Tang, R., Zhang, X., Ma, X., Lin, J., & Ture, F. (2024). Found in the middle: Permutation self-consistency improves listwise ranking in large language models. *Findings of the Association for Computational Linguistics: NAACL 2024.*

Wang, P., Li, L., Chen, L., Cai, Z., Zhu, D., Lin, B., Cao, Y., Liu, Q., Liu, T., & Sui, Z. (2023a). *Large language models are not fair evaluators.*

Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E., Narang, S., Chowdhery, A., & Zhou, D. (2023b). Self-consistency improves chain of thought reasoning in language models. *International Conference on Learning Representations (ICLR).*

Wang, Y., Ma, X., Zhang, G., Ni, Y., Chandra, A., Guo, S., Ren, W., Arulraj, A., He, X., Jiang, Z., Li, T., Ku, M., Wang, K., Zhuang, A., Fan, R., Yue, X., & Chen, W. (2024). MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track.* arXiv:2406.01574.

Wataoka, K., Takahashi, T., & Ri, R. (2024). *Self-preference bias in LLM-as-a-judge.*

Yang, G., Liu, Y., Miao, S., Liang, X., Liu, Z., & Huang, H. (2025). *Llms judge themselves: A game-theoretic framework for human-aligned evaluation.*

Young, H. P. (1988). Condorcet's theory of voting. *American Political Science Review*, 82(4), 1231–1244. `https://doi.org/10.2307/1961757`

Young, H. P. & Levenglick, A. (1978). A consistent extension of Condorcet's election principle. *SIAM Journal on Applied Mathematics*, 35(2), 285–300. `https://doi.org/10.1137/0135023`

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems (NeurIPS)*.